International Journal of Wireless and Microwave Technologies, **2026, 3, 111-127**
Published Online on June 8, 2026 by MECS Press (http://www.mecs-press.org/)
DOI: 10.5815/ijwmt.2026.03.08

An Example of Developing a High-level Requirements Specification for AI-based Software using ChatGPT

Sergey Orekhov*
Department of SEMIT, National Technical University «KPI», Kharkiv, Ukraine
E-mail: sergey.v.orekhov@gmail.com
ORCID ID: https://orcid.org/0000-0002-5040-5861
*Corresponding Author

Pavel Taran
Department of SEMIT, National Technical University «KPI», Kharkiv, Ukraine
E-mail: taranpaher@gmail.com
ORCID ID: https://orcid.org/0009-0006-6624-2514

Nikuta Bahatskyi
Department of SEMIT, National Technical University «KPI», Kharkiv, Ukraine
E-mail: Nikita.Bahatskyi@cs.khpi.edu.ua
ORCID ID: https://orcid.org/0009-0008-2773-1600

Received: 02 March, 2026; Revised: 15 April, 2026; Accepted: 26 April, 2026; Published: 08 June, 2026

Abstract: Over the past seven years, significant changes have occurred in both the development paradigms and the practical use of software systems of varying complexity. These changes are largely driven by the rapid adoption of online artificial intelligence technologies based on large-scale language models. Such models are currently actively used in software development tasks, including source code generation and test plan creation, thereby integrating across various stages of the software development lifecycle. This article examines a classic research object—namely, the process of developing a system requirements specification—and proposes an approach to its formal verification using the ChatGPT online service. First, a detailed mathematical formalization of the research object is presented, followed by a structured model for preparing system requirements in projects using ChatGPT at various stages of development. Next, the proposed approach is illustrated using a real IT project example, demonstrating the sequential stages of requirements preparation in a modern development environment. The article defines the main categories of system requirements and discusses their representation in project documentation. To support the analysis, relevant tabular data and UML diagrams are provided. Furthermore, the study describes a methodology for formal requirements verification through prompt-based interaction with the ChatGPT system. The scientific novelty of this work lies in the application of requirements verification by modeling the expected behavior of the future software system using ChatGPT. Future research directions include incorporating a fifth category of requirements – business rules – using ChatGPT, which will enable modeling the behavior of the software system in real business processes.

Index Terms: System requirements specification, systems requirements engineering, formal verification, chatGPT

1. Introduction

A System Requirements Specification (SRS) constitutes one of the fundamental artifacts within the software development life cycle [1–2].

The overall quality of requirements significantly influences subsequent stages, including system design, implementation, testing, and maintenance.

1.1. Problem Statement

In recent years, large language models have been increasingly applied to automate tasks related to the analysis, structuring, and formalization of system requirements. However, existing approaches to their integration into engineering workflows are predominantly described at a conceptual or procedural level, lacking rigorous formalization. The absence of a well-defined formal model limits the ability to objectively assess quality attributes, ensure reproducibility of results, and systematically incorporate language models into established software engineering processes.

1.2. Motivation

Under these conditions, the development of a mathematical model describing the generation of SRS with the involvement of a language model as an intelligent component becomes a relevant scientific problem. In parallel, it is necessary to address associated challenges, including the formal verification of requirements through behavioral modeling and the creation of algorithmic support for synthesizing diverse categories of requirements in accordance with customer specifications.

Accordingly, the objective of this study is twofold. First, it aims to construct a formal mathematical model of the SRS generation process based on user-provided data and domain-specific contextual information, with a language model serving as the core generative mechanism. Second, based on the proposed model, the study seeks to develop an algorithm for automated requirements generation, accompanied by a structured procedure for their formal verification through simulation-based modeling.

To achieve this objective, we introduce an abstract mathematical framework that describes the transformation of initial user inputs and contextual parameters into a structured SRS. The proposed model explicitly accounts for the stochastic properties inherent in language models, as well as the potential involvement of human experts in decision-making and validation processes. Such consideration ensures both theoretical completeness and practical applicability within real-world software engineering environments.

2. Related Works

A review of recent publications, particularly from 2023 onward, indicates several notable tendencies in the application of generative artificial intelligence within requirements engineering.

2.1. Generative Artificial Intelligence (GenAI) and Error Sensitivity

At first, Generative AI technologies are increasingly employed to assist in the preparation of software requirements specifications, enabling on-demand generation of program code fragments and unit testing instructions. In addition, such systems facilitate the automated creation of UML diagrams and other abstract graphical representations that support system design. Second, these developments reflect a broader capability of generative models to interpret textual descriptions of user requirements and transform them into corresponding program code [3–5].

These observations suggest that generative AI can, in principle, be applied to produce a wide range of software development artifacts. As a result, scholarly and practical interest in this direction of requirements engineering has grown significantly. The outputs of such systems may include source code in selected programming languages, diagrams with varying levels of detail, multi-level test plans, and design artifacts suitable for subsequent deployment in cloud environments.

Despite the demonstrated effectiveness of generative models in producing high-level requirements descriptions, their use may still introduce inaccuracies, ambiguities, or unintended biases, which pose risks to software reliability and quality. Moreover, generative systems are sensitive to the characteristics of the data on which they were trained. Empirical studies indicate that models trained on data containing flawed judgments or incorrect conclusions may reproduce similar deficiencies in newly generated design decisions. Consequently, the principle of critical verification remains essential when employing such tools.

2.2. Prompting for Language Model (LM)

In this context, the formulation of precise prompts becomes a critical factor in obtaining reliable outputs from generative systems. Prompts define the textual input that guides the generation of requirements specifications. While the internal training process of generative models cannot be directly controlled, attention can be focused on improving the structure and clarity of the prompts provided to them. Accordingly, this study proposes the development of an abstract mathematical model that formalizes the transformation of initial user requirements and contextual information into a structured software requirements specification. The proposed model accounts for the probabilistic nature of language models and incorporates the potential role of human oversight in the decision-making process [3–5].

3. System Model

3.1. Overview

Let us assume that we are designing a software system that uses language models. Then, when creating a formalized system requirements specification for this system, we need to transform unstructured or weakly structured user requirements into a stable, clear structure. Let us describe this transformation process as a sequence of mathematically defined mappings. We will introduce the first mapping as follows: $X \to S$ is a set of input data, and S is the final specification of system requirements.

We define inputs for STS building as:

$$X = (R, D, C) \tag{1}$$

Where
$R \in \{0,1\}^n$ = many user requirements and requests
$D \in \{0,1\}^m$ = a set of documents in the subject area (descriptions of business processes, regulatory documentation)
$C \in \{0,1\}^k$ = a set of contextual constraints (standards, security constraints, specification language)

The output is a system requirements specification:

$$S = \{S_1, S_2, .., S_L\} \tag{2}$$

Where
$s \in \{0,1\}^L$ = represents a formalized requirement, classified into one of the following types: business, functional, non-functional, user stories, or business rules (constraints) [1-2]

Let us assume that we are preparing a requirements specification using a language model. Then we need to formulate an input query for this model.

3.2. Model Components

The first step involves aggregating and structuring the input data into a single text query (prompt):

$$P = f_p(R, D, C) \tag{3}$$

To describe requirements, the language model (LM) will be represented as a parametric function:

$$LM_Q: T \times K \to T \tag{4}$$

Where
T = space of texts
K = generation context (system instructions)
Q = parameters of model

In this case, the results of requirements generation are presented as follows:

$$R_{uf} = LM_Q(P, K) \tag{5}$$

Where
R_{uf} = a set of generated requirements in an unstructured form

To improve the quality of requirements, validation and evaluation functions are introduced:

$$R_{qr} = f_v\left(f_q(R_{uf})\right) \tag{6}$$

Where
f_q = the function of assessing the quality of requirements according to the criteria of completeness, ambiguity and verifiability
f_v = compliance validation function (for example, ISO/IEC 29148)

The final specification is generated by the structuring function:

$$S = f_s(R_{qr}) \tag{7}$$

Where
f_s = classifies requirements and presents them in the form of a structure of SRS

The final stage is a formal comparison of various specification options. For this, a quality objective function is introduced:

$$Proof(SRS) = aProof_p + bProof_u + cProof_t \tag{8}$$

Where
$Proof_p$ = completeness indicator
Proofu = unambiguity indicator
Prooft = raceability indicator
a, b, c = weighting coeficients

We will maximize the function (8).

3.3. Description

The proposed model (1)-(8) can be expanded by taking into account the stochastic nature of the language model, introducing iterative dialogue with an expert (human-in-the-loop), and representing requirements as a directed dependency graph or UML diagrams [6]. This allows the model to be used in decision support systems for requirements analysis.

Clearly, it is impossible to implement all five functions described above in a single study. Therefore, in our work, we focused on solving the problems hidden in formulas (1)-(8). The issue of optimization (8) is not considered in this paper. Therefore, we will now describe the algorithm for solving problems (1)-(8) in a single, so-called package of actions.

4. Proposed Approach

4.1. Conditions

When solving problems (1)-(8), it is necessary to prioritize the solution. To determine these priorities, we should first consider the current trends in the IT development market.

First, the customer demands results as quickly as possible. This means creating a software product with basic functionality to quickly receive initial feedback from potential users of the IT product.

Second, the first condition places the developer in a situation where they must prove the effectiveness of the future IT solution already at the requirements verification stage [7-8]. This means, for example, that the complete version of the software code does not exist, but we need to demonstrate to the customer that this software code will be successful. This applies to formula (6).

Third, according to formula (5), we must transform unstructured texts into a set of UML diagrams, which then constitute our requirements specification artifact. This transformation (5) is associated with the risk of misinterpretation of the base texts by the user. Such transformations can be implemented using language models, both manually and automatically, to speed up the process of preparing the requirements specification [9].

Thus, we are required to formulate a system requirements specification under the following conditions: a) rapid development, b) the use of language models, and c) formal verification of the base functionality.

To address the three requirements outlined above, we propose developing prompts according to formula (3), which will generate the main types of requirements in the chatGPT system [10-13].

4.2. Idea

We propose the following plan (stages) for describing requirements. It includes the following stages:

1. **Introduction:**
   4-5 sentences describing the relevance of the software system development.
2. **Problem statement:**
   4-5 sentences outlining the need our development addresses for the end user.
3. **Proposed IT solution:**
   3-5 sentences describing the final development outcome.

4. **IT solution format:**
   4-5 sentences describing the technological basis for development.
5. **Monetization:**
   4-5 sentences describing how the software system's output can be monetized.
6. **Project methodology**:
   3 sentences describing the development life cycle: agile, scrum etc.
7. **Business requirements**:
   Unlimited text length.
8. **Functional requirements**:
   Unlimited text length.
9. **Non-functional requirements**:
   Unlimited text length.
10. **User stories**:
    Unlimited text length, but includes UML diagrams.
11. **Software system architecture**:
    UML deployment diagram.
12. **Mock up**:
    A drawing of the application's main screen.

As we can see, this plan allows us to create a custom chatGPT prompt system for each stage. Let's list the main requirements for such prompts for the stages mentioned above. Let's start with relevance.

We recommend formulating relevance in the following style. The first two sentences are needed to indicate the scope of the future software system. The third sentence indicates the unsolved problem that our software product will address. The last sentence indicates the development goal on a global scale.

When describing the problem, we should indicate in two sentences what software product we need to develop. The main question is what effect this software product will produce.

In the "Proposed IT Solution" section, we need to describe the business process in which our software product is involved in three to four sentences. We should also demonstrate the expected outcome of this business process.

In the "IT Solution Format" subsection, we need to specify the basic software platform (form) on which the IT solution will be implemented in three sentences. It is also advisable to demonstrate the IT solution format.

In the "Project Methodology" section, we need to justify the choice of software development life cycle. Also, if possible, indicate which stages of the life cycle should be emphasized.

The next key section is business requirements. Here, we will assume unlimited text space. However, each requirement item has the following format: <BR #> Name: text. Each requirement has a unique number and title. The text is described as follows. It is necessary to manipulate user and program objects. Therefore, we formulate the text as follows: the user (program) has the ability to perform the following action. This means that rule (5) can be applied here. It can then be improved using rule (6).

The next section of our specification consists of functional requirements. Like business requirements, they are described using a structure like: <BR # - FR #> Name: text. Here, we clearly tie a function to a specific business requirement. Each functional requirement is formulated as follows: the user can do this with that result. This means that the functional requirement includes the type of user of the software system, their specific action, and the result they should achieve.

When describing non-functional requirements, we rely on the SWEBOK model and the ISO 9126 standard [1-2]. It identifies six software quality factors with associated quality criteria and metrics. The requirement structure looks like this: <BR # - NFR #> Factor (Criterion) Name: text. The text specifies how the software system should function. Therefore, it is also advisable to indicate the connection to the functional requirement.

The subsequent component of the specification consists of user stories, which represent one of the most structurally complex elements, as they integrate textual test descriptions with UML-based visual representations. Each user story follows a standardized format:

<US # (FR #)> Name: text. Acceptance Criteria: 1) … 2) …

Every user story includes a title and a descriptive section that articulates the objective the user intends to achieve through interaction with the software system, specifying the expected system behavior. This is followed by a set of acceptance criteria that define the conditions under which the task is considered successfully completed from the user's perspective. These criteria effectively describe the system responses or actions that indicate fulfillment of the intended functionality. Additionally, each user story incorporates a tabular representation, the structure of which is presented in Table 1.

Table 1 summarizes the principal components of a user story, which are subsequently visualized using a use case diagram, as shown in Figure 1.

Table 1. User Story (Example)

Element	Details
ID	UC 001 (FR001)
Use case	Adding, changing and viewing accounts of WEB users
Initiator	Platform administrator
Participants	Administrator and software system
Prerequisites	Administrator must log in to the system
Main stream of actions	1. Administrator activates the list of WEB users 2. The system provides a form of entering the data about Web user 3. The administrator enters the data about WEB user and sets the user rate 4. The administrator confirms the accuracy of user data 5. The system saves the record and updates the overall user rate 6. The system displays the updated list of users
Alternative streams (optional)	1a. The administrator has not confirmed the data entry, that the system does not allow to save the last record.
Post-conditions	The list of users is available for viewing by administrator and users.
Exceptions (optional)	1. Error of DB connection was happened, when administrator has been adding the data entry.

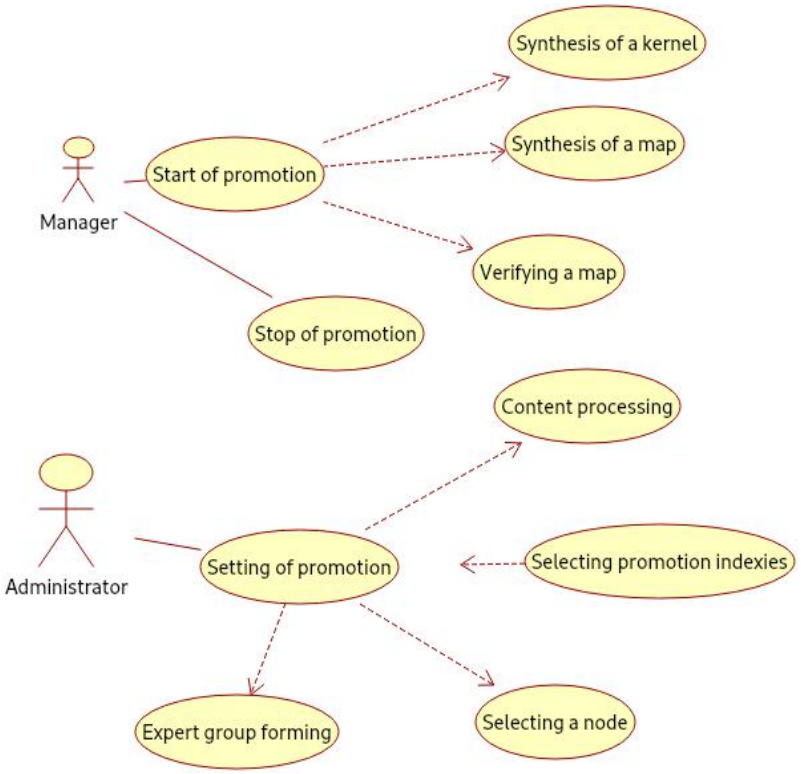

Fig. 1. Use cases based on proposed user Story (Example)

The next section of the specification describes the main interface of the prospective software system, an example of which is provided in Figure 2. For illustrative purposes, this study deliberately presents specification elements derived from three distinct IT projects completed previously (Figures 1–2). Despite originating from different contexts, all examples adhere to a unified structure for requirements specification, formalized according to formulas (1)–(8). A common feature across these projects is that the formulated requirements were validated using ChatGPT. The methodological recommendations outlined above can be effectively transformed into structured prompts for interaction with a LM such as ChatGPT.

Thus, in accordance with formulas (1)–(8) and the methodological recommendations outlined earlier (Table 1 and Figures 1–2), a semi-structured representation of software requirements can be constructed. The resulting combination of textual descriptions and graphical models provides a suitable basis for applying a language model to the task of requirements verification. For this purpose, it is necessary to prepare a well-defined set of verification prompts that guide the evaluation process.

A more advanced strategy would involve the development of a dedicated repository containing examples of both successful and problematic requirements specifications derived from previously completed projects.

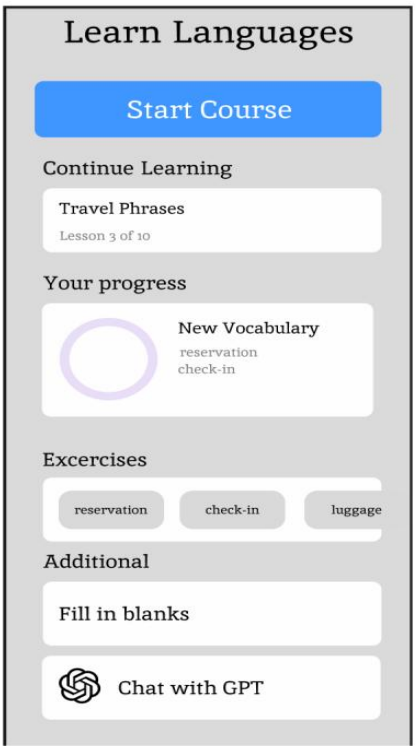

Fig. 2. Mock Up (Example)

The proposed repository could be used to train a language model capable of both generating requirements and performing subsequent verification. However, in practical conditions this approach is difficult to implement effectively, since empirical observations indicate that hidden defects in requirements specifications may account for up to 50% of their total content. As a result, software developed on the basis of such imperfect data would likely demonstrate unsatisfactory quality.

4.3. Algorithm

Accordingly, two principal strategies can be distinguished. The first assumes that a business analyst formulates the requirements, after which a language model is employed to verify their correctness and consistency. The second strategy involves the use of a language model to generate requirements, which are then reviewed and validated by a business analyst through a formal requirements audit. In this study, preference is given to the first strategy. A simplified representation of this process is provided in Figure 3.

The proposed approach therefore combines expert analytical work with the capabilities of a language model. The analyst prepares structured prompts that guide the model in generating an initial version of the requirements. Subsequently, the analyst evaluates the semantic accuracy of each requirement and develops the corresponding diagrams. In contemporary practice, the creation of diagrams and the configuration of the language model can be partially outsourced. Additional prompts are then prepared to simulate the operation of the intended software system. The verification procedure itself may be recorded as a video demonstration, allowing stakeholders to observe the expected system behavior. An illustrative example of preparing a requirements specification according to the proposed algorithm is presented below.

Let's look at the application of our problem-solving algorithm (Figure 3) using a specific example.

5. Experiments

5.1. Goal

The goal of the experimental development was to develop a SRS for software using the ChatGPT online service, which helps solve the problem of generating pet feeding recipes. This software should integrate data on the animal's current body needs, the owner's budgetary constraints over time, and the owner's personal preferences.

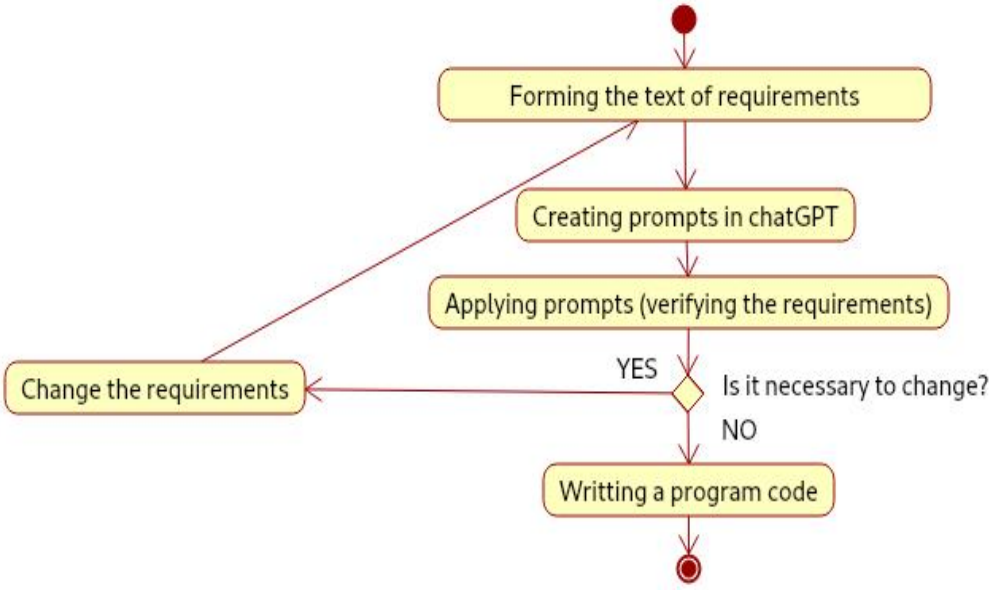

Fig. 3. Activities in our approach

The result will be an automatically generated recipe. Such a recipe could potentially also be used for selecting pet food. However, in this article, we will examine the verification process for such requirements and how to implement formal verification in accordance with our proposals above.

5.2. Elements of SRS

To begin, we'll present the requirements themselves in a brief summary with corresponding UML diagrams:

1. **Introduction:**
   As demonstrated above, there is a pressing need to create a web application that will help plan pet care, generate customized feeding recipes based on animal characteristics (age, weight, activity, breed), and provide reminders for key events: meals, walks, veterinary visits, parasite treatments, or grooming appointments.
2. **Problem statement:**
   Most pet care apps are either reminder calendars or feed calculators. They don't integrate personalized feeding recipe generation using intelligent recommendations. The user must search for the optimal diet, check the nutrient content, and ensure that the required nutrients, macro- and microelements, and vitamins are met.
3. **Proposed IT solution:**
   Therefore, we propose building the software using chatGPT language model [7-10]. We will use it as an intelligent advisor to create personalized pet feeding recipes. Our system will be based on a deep analysis of individual animal characteristics.
4. **IT solution format:**
   The intelligent web application will allow users to maintain a profile for each animal (age, species, weight, health, activity, food type, etc.). It will provide reminders about feedings, walks, veterinary visits, parasite treatments, and grooming procedures. ChatGPT API helps us generate feeding recommendations or new recipes based on the animal's characteristics. The mobile version displays unobtrusive advertising, while the web version displays contextual advertising to support the free subscription.
5. **Monetization:**
   The user receives free access to basic functionality, which includes: creating one pet profile, using basic reminders and an event diary, and creating basic recipes via chatGPT.
6. **Project methodology**:
   Agile methodology (a flexible development model) was chosen for this IT project. This approach allows for the gradual development of a software product, dividing the process into short iterations (sprints), during which individual functional modules are implemented. After each sprint, testing is conducted, a user demonstration is conducted, and feedback is collected, which is used to plan the next phase. Agile also promotes flexible management of priorities – for example, you can implement basic reminder functionality first, and then add advanced features, such as automatic meal planning or health statistics.

Paid consultations with a veterinarian are also available, allowing the user to book a one-time online consultation with a veterinarian without having to navigate to other apps. The web app acts as an intermediary, receiving a commission (a percentage) on the consultation fee.

A premium subscription is also available, which provides access to advanced functionality. This includes: unlimited viewing of multiple pet profiles, no ads, several free monthly consultations with a veterinarian, personalized weekly or monthly diet plans, and priority user support.

5.3. Experimental Data

The first step in preparing a system requirements specification is to define the business requirements (BR) (Table 2) [6].

The functional requirements for a web application are listed in Table 3, and the non-functional requirements (NFR) are listed in Table 4 [1-2].

Table 2. Sample of Business Requirements

No	Name	Content
BR001	Animal registration and profiling	The system shall provide the user with the capability to create an animal profile by specifying fundamental parameters, including species, breed, age, weight, activity level, and health condition.
BR002	Reminders about care events	The system shall generate scheduled reminders related to feeding, walking, and veterinary examinations in accordance with predefined timelines.
BR003	Formulating a feeding recipe	The system shall utilize ChatGPT to produce individualized feeding recommendations or dietary recipes based on the recorded characteristics of the animal.

Table 3. Sample of Functionality

No	Name	Content
FR001 (BR001)	Creating an animal profile	The system shall enable the user to create an animal profile, upload an image, and specify essential attributes such as species, weight, age, health status, and activity level.
FR002 (BR002)	Creating notifications	The user shall be able to record events (e.g., feeding, walking, veterinary visits) with associated time, date, and recurrence parameters.
FR003 (BR 003)	Recipe generation via chatGPT	The system shall transmit the animal's data to the ChatGPT API and obtain a personalized dietary recommendation or recipe in response.

Table 4. Sample of Quality Requirements

No	Name	Content
NFR001 (BR001–BR005)	User-friendly interface	The user interface shall be designed in accordance with principles of intuitiveness and adaptability, ensuring ease of use across different user contexts and devices.
NFR002 (BR002)	Reliability of notifications	The system shall deliver reminders with minute-level precision, regardless of the application state, including background operation or inactive (closed) modes.
NFR003 (BR003)	Recipe generation productivity	Requests to the ChatGPT service shall be processed within a maximum time threshold of 5 seconds; in the event of a failure, the system shall automatically initiate a retry mechanism.

The data from Tables 1-3 allowed us to move on to designing user stories.

US001 (FR001) – creating a pet profile: As a user, I want to create a profile for my pet to receive personalized feeding recommendations and care reminders.

Acceptance criteria:

1. User can enter data: name, species, breed, age, weight, health status, activity level.
2. System checks the correctness of the entered data (for example, weight > 0).
3. After clicking "Save", the pet profile is saved in the system.
4. If the user has a free subscription, the system allows only one profile.

US002 (FR002) – create a reminder: As a user, I want to create reminders for feeding, walking, veterinary visits, parasite treatments, and grooming visits so that I don't forget about my pet's daily care.

Acceptance criteria:

1. User can select the event type (feeding, walking, veterinarian).
2. The date, time, and recurrence of the event can be set (daily, weekly, etc.).
3. System sends a push or email notification at the specified time.
4. User can edit or delete the reminder.

US002 (FR003) – feed recipe generation via chatGPT: As a user, I want to receive a feed recipe for my animal to know the optimal amount and composition of food according to its parameters.

Acceptance criteria:

1. User clicks “Create recipe”.
2. System sends the animal profile data to the chatGPT API.
3. User can mark the event as completed or postpone it.
4. Response contains a description of the diet, portion size, product recommendations.
5. User can save the recipe in the profile.

The listed user stories can be combined into a single use case diagram – Figure 4.

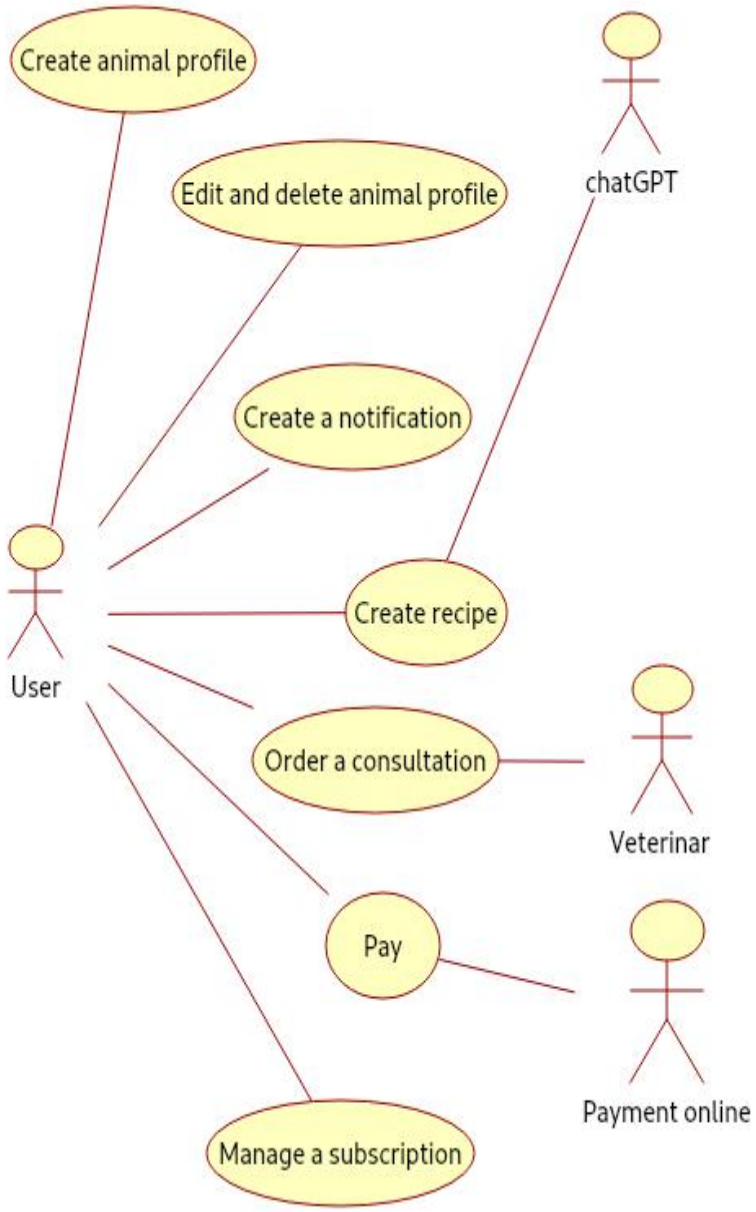

Fig. 4. Use cases of our approach

Based on user stories, we also propose a visual representation of the main page of the web application – Figure 5. Next, Figure 5 shows the page for creating a dog feeding recipe. And also the architecture of the future software product – Figure 6.

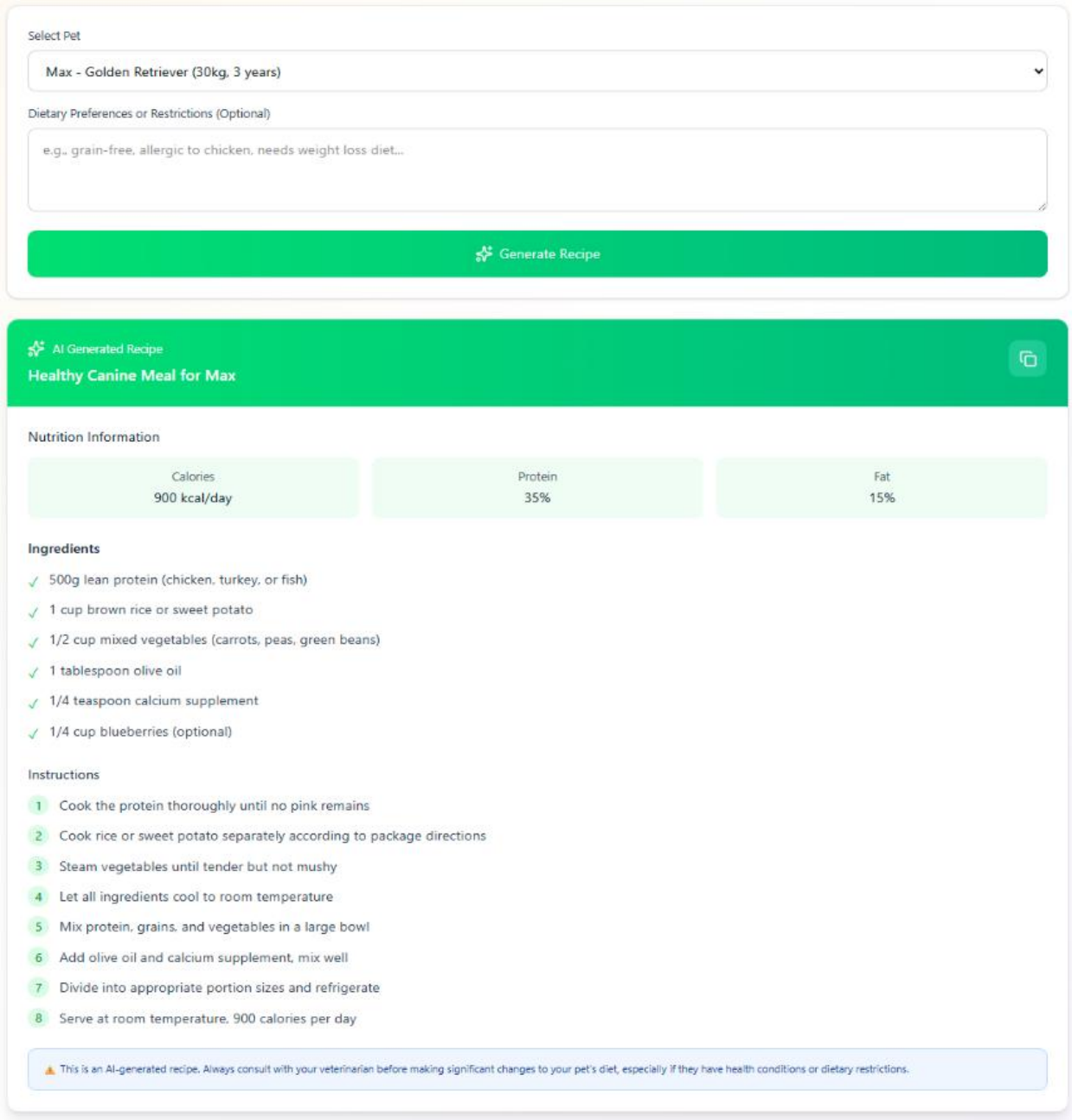

Fig. 5. Create a recipe

Thus, we have obtained almost all the necessary data for verification. In our case, we propose simulating the work to obtain the desired primary output of the future software system – the recipe. Therefore, Figure 7 shows the recipe prompt that we will request from chatGPT.

Let's decipher the prompt. First, we need to clarify our task to LM. We propose the following text: “You are a pet nutrition expert with a veterinary qualification and a pedagogical approach to teaching. Your task is to generate a structured, practical, and safe diet or feeding recipe for a specific animal based on the information I provide. Important! This recipe is a recommendation. Your pet's current medical conditions should also be taken into account.”

The next block of the prompt is the input data. We suggest the following components:

1. Species: {kind} (e.g., cat, dog, rabbit).
2. Breed: {breed}.
3. Age: {age}.
4. Weight: {weight}.
5. Medical indications: {indexes} (short: allergies, chronic diseases, or simply the absence of them — none).
6. Preferences and restrictions: {preferences} (text of any length where we indicate food preferences. For example, my cat prefers dry food).

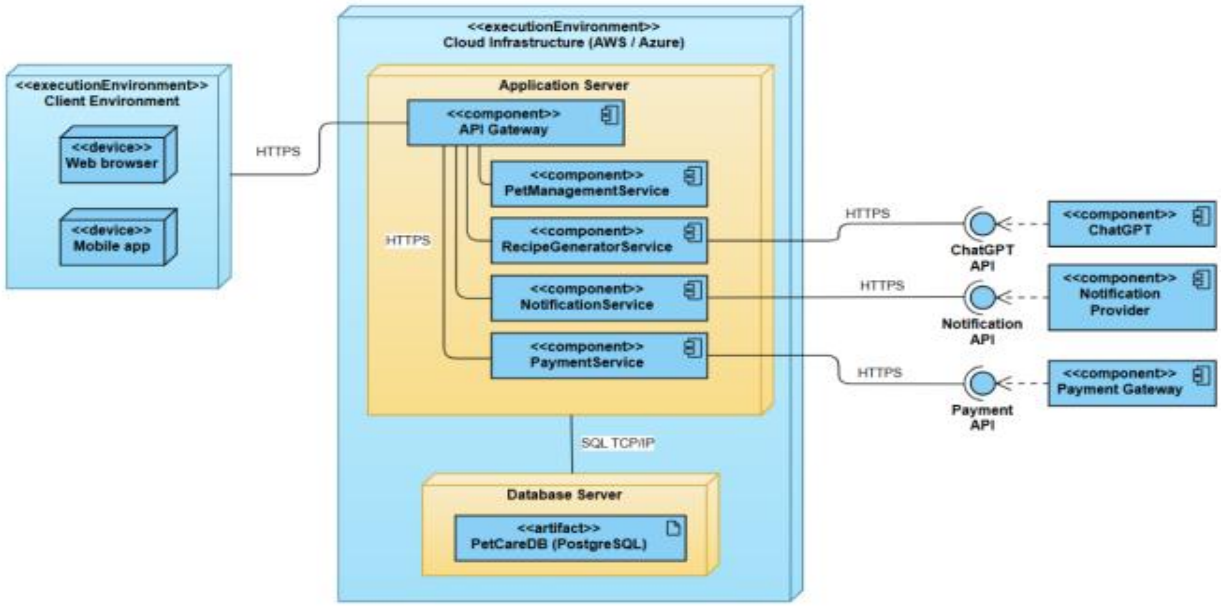

Fig. 6. Deployment diagram

Let's draw up recommendations for the LM response. Response Limitations and Conditions are:

1. The response consists of two sections: a) a user-friendly text section and b) a machine-readable section in JSON format. The JSON response format is shown below.
2. The maximum length of the short response is 300-400 words.
3. In the short response, we use headings, brief bullet points, emoticons, and clear numerical estimates and recommendations (g/day, calories).
4. The JSON response file must contain the following fields: summary, daily_calories, calculation_notes, meals, ingredients_list, warnings, and references. The JSON does not contain additional text.
5. Specify how calories are calculated (shortcut formula/coefficient), as well as the source from which the coefficients are calculated.
6. Specify generation parameters (API recommendation): 'temperature=0.2', 'max_tokens=600'.

User Response Format (required):

1. Summary (3-4 sentences).
2. The maximum length of the short response is 300-400 words.
3. Approximate daily calorie intake and general recommendation (g/day, calories).
4. Number of meals per day.
5. Sample menu/recipe for one day, broken down by meals (exact grams).
6. List of recommended ingredients and possible substitutions.
7. Health complication warnings and medical restrictions.

JSON file format is:

```
{
"summary": "...", "daily_calories": 0, "calculation_notes": "...",
"meals":[
 { "name": "Morning",
   "weight": 0,
   "components":
    [{"ingredient": "chicken",
      "weight": 0,}],
      "calories": 0 }]
      "ingredients_list": ["ingredient1,..."],  "warnings": ["..."],  "references": "...",
}.
```

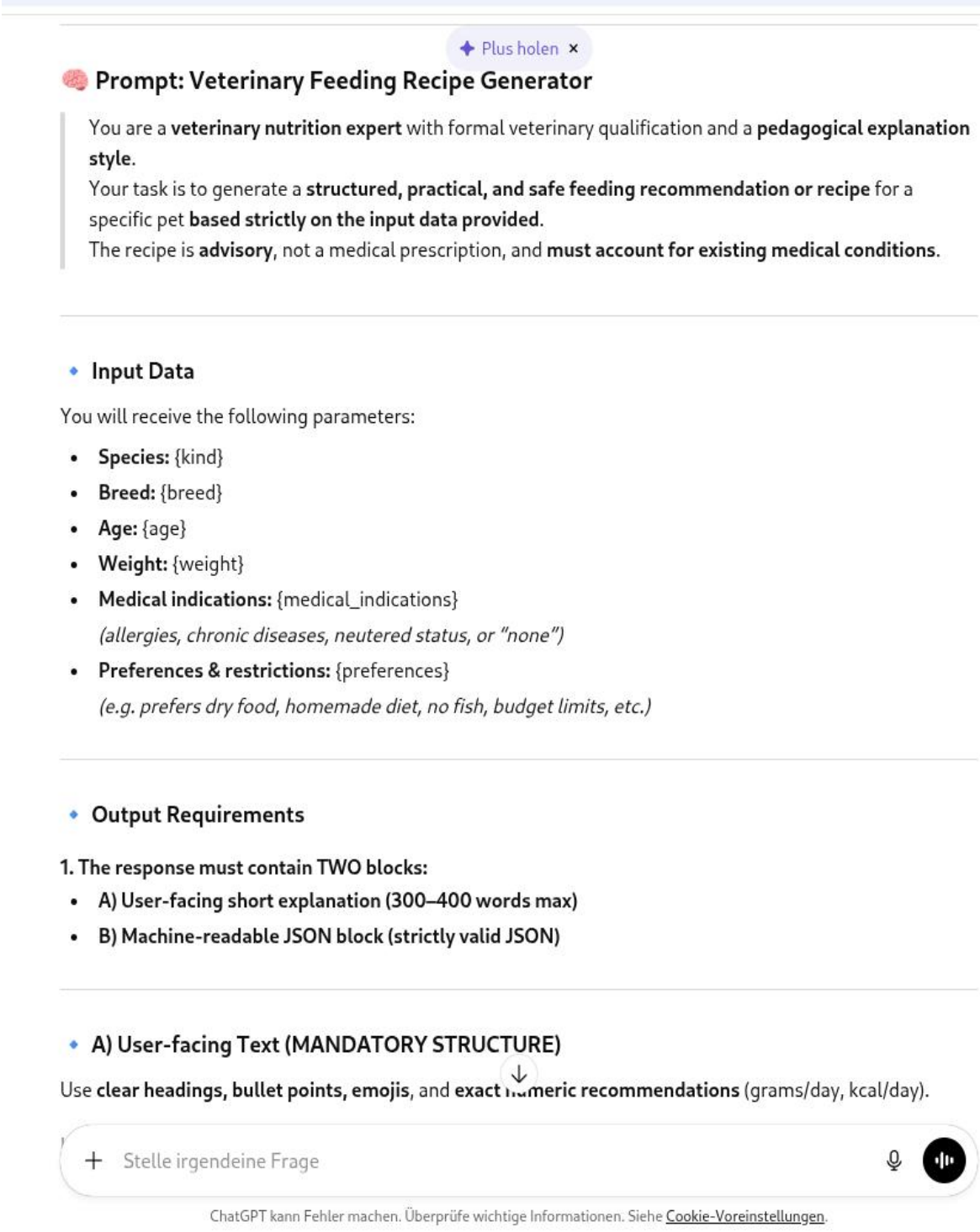

Fig. 7. Prompt to create a recipe

Thus, we are ready to interact with the LM. At the stage of manual verification, the API request to ChatGPT was replaced by a manual copy of the prompt from an open ChatGPT session, after which the structured response was removed. The results for the test is shown in Figure 8.

We have prepared the following request: “Species: cat Breed : Britannic, Age : 8 years old, Weight: 5 kg, Medical indications: fat cat, Preferences: dry feed”. The answer of chatGPT is depicted in Figure 8.

The results obtained fully met the requirements formulated in the functional model of the system. In particular, the possibility of dynamic generation of the diet based on the minimum set of user data was confirmed, the consistency of the response structure with the formats provided for by the architectural solution, the presence of mechanisms for controlling the correctness of the entered parameters, and the compliance of the information content with basic veterinary principles and dietary standards.

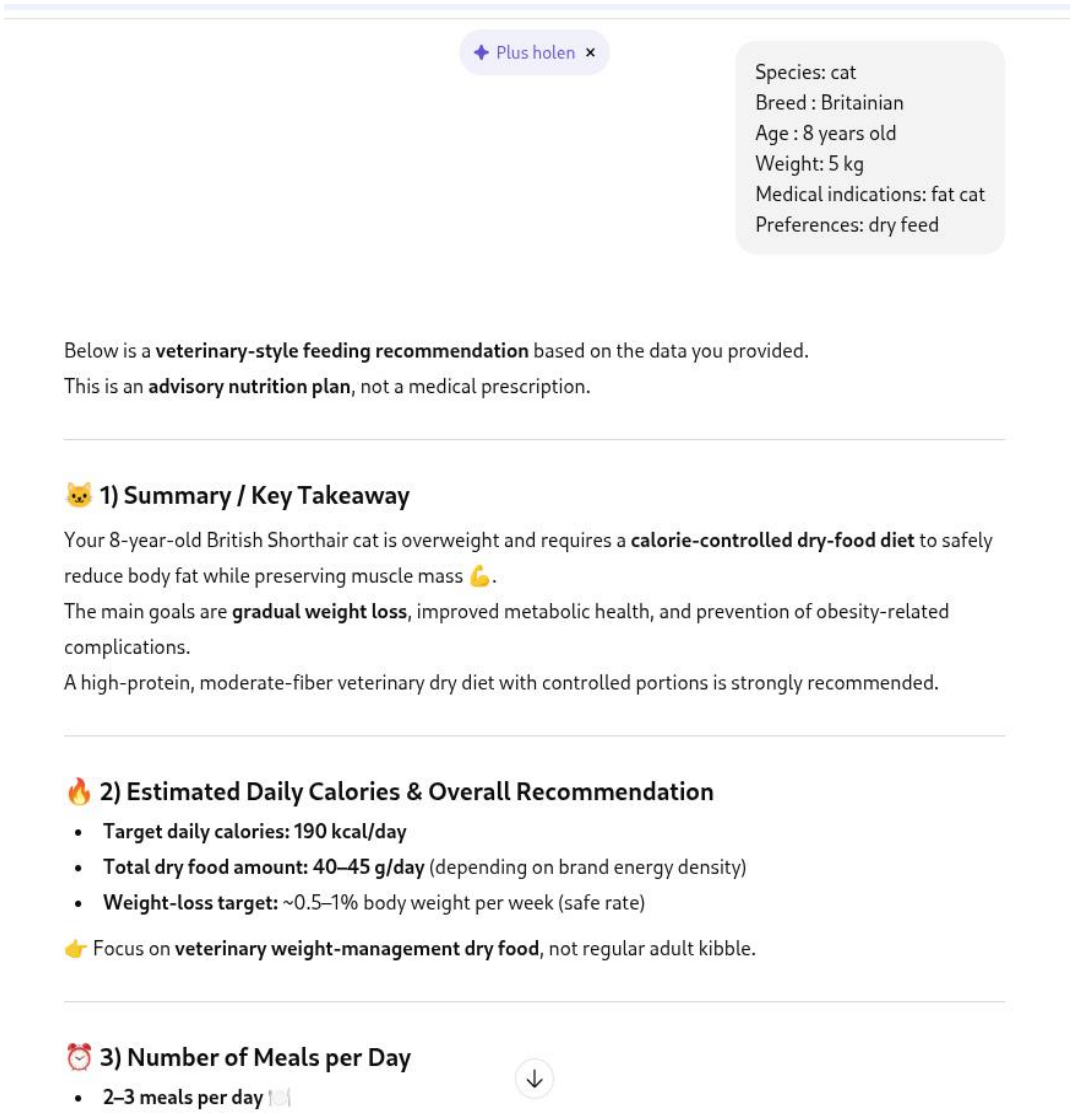

Fig. 8. Generated recipe

Thus, the verification confirmed that the algorithm described in the project documentation can be implemented within the given architecture without the need for its revision or complication.

As a result of the above steps, we are able to test a hypothesis about the functioning of our software product, which is based on a language model. As part of this test, we are also able to refine, for example, the input data (Figure 7) and even simulate the language model's response to this input (Figure 8).

6. Discussion

To evaluate the obtained results, a set of scholarly sources was examined that had been recommended by ChatGPT in response to a query aligned with the topic of this study. In this way, generative AI was intentionally incorporated into the research and writing process itself. The analysis of these publications highlights an important observation: for practitioners, generative models largely remain a “black box,” which necessitates a clearer understanding of effective interaction strategies. A critical factor in this interaction is the formulation of well-structured queries. When a business analyst follows a clearly defined procedure for preparing a software requirements specification, this structure can be directly reflected in the prompts provided to the generative model. Precisely formulated prompts, in turn, contribute to obtaining more reliable and higher-quality outputs. Accordingly, this article presents both a structured algorithm for the analyst’s work and examples of prompts intended to improve the quality of the resulting specification.

The quality of a requirements specification cannot be adequately assessed by superficial metrics such as its length or level of detail. Instead, it is more appropriately evaluated through the number of defects revealed at the software testing and verification stages. In this context, generative AI can again be employed as a supportive tool. By using targeted prompts and structured question–answer interactions, requirements verification can be performed, as illustrated in Figures 7–8.

7. Conclusion

The methodology proposed in this study exhibits both notable strengths and inherent limitations. One of its primary advantages is the introduction of an abstract mathematical framework that formalizes the process of system requirements generation. This framework provides a structured representation of the key stages involved in developing a requirements specification and clarifies the relationships between them. The practical applicability of the model is demonstrated through a real-world case study in which requirements for an operational system were formulated with the assistance of ChatGPT.

A second important contribution is the incorporation of a large language model, namely ChatGPT, as an operational instrument for implementing the proposed framework. The study presents examples of carefully structured prompts that explicitly define input conditions and expected outputs, as well as prompts designed to simulate the anticipated behavior of the target software system. This illustrates how the theoretical framework can be translated into practical engineering procedures.

Third, the approach emphasizes iterative collaboration between a business analyst and the language model during both the requirements elicitation and verification stages. Such interaction supports the clarification of ambiguities, refinement of requirement statements, and validation of logical consistency, thereby increasing the robustness and reliability of the final specification. Verification is performed through simulated scenarios reflecting the expected operation of the software system.

Fourth, the study outlines a complete software design cycle that encompasses requirements engineering, formal verification, and the derivation of program code as the final outcome (Figure 9).

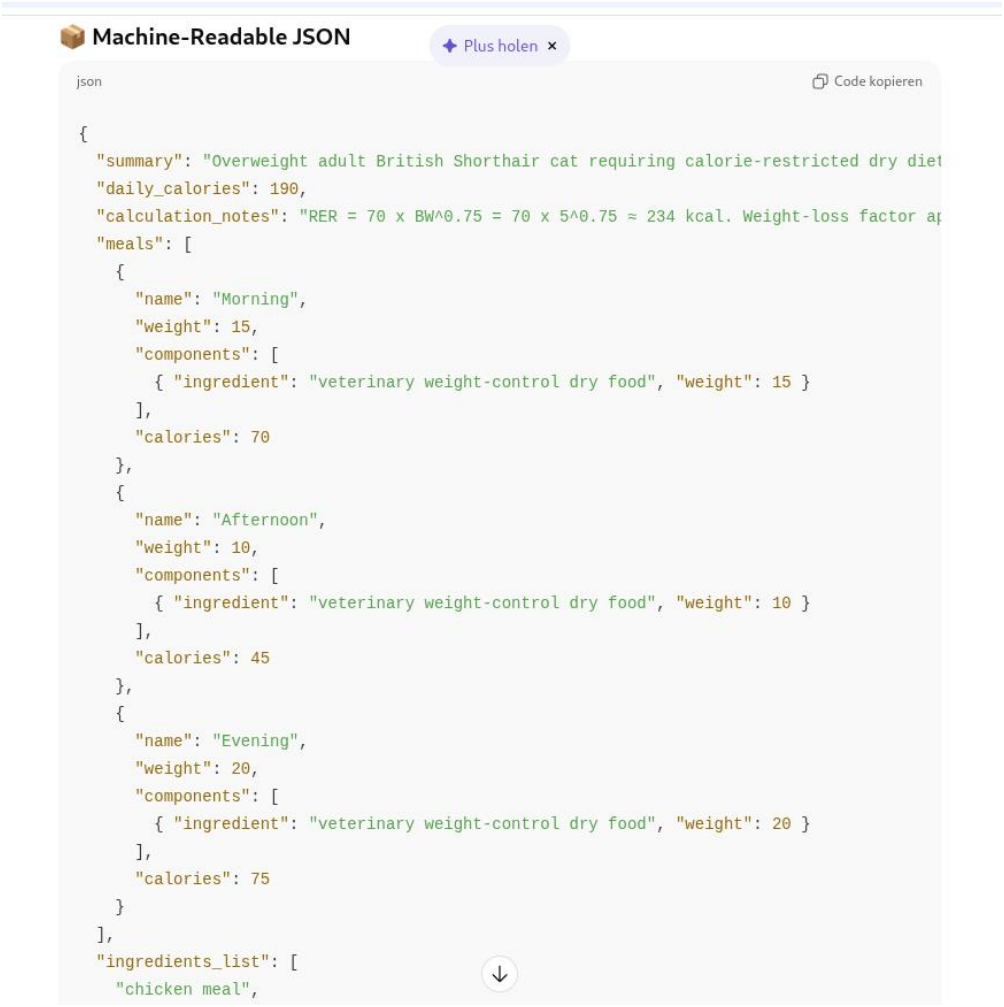

Fig. 9. JSON Content

At the same time, the methodology has certain limitations. In particular, classical mathematical programming techniques were not employed to solve models (1)–(8). Moreover, the study does not include a comparative analysis between the proposed approach and traditional optimization-based methods with respect to efficiency, accuracy, or

computational performance. The absence of such benchmarking constrains the possibility of quantitatively evaluating the relative advantages of the developed framework.

All the Declarations and Statements

Author Contributions Statement

S. Orekhov – Conceptualization, Supervision: Proposed research ideas, Constructed the overall framework, and supervised project execution.
P. Taran – Prepared the initial version of the manuscript, contributed to the review of related literature, and documented the technical foundations of the study.
N. Bahatskyi – Drafted the initial manuscript, contributed to the literature survey, and documented the technical background of the study.

All authors have read and agreed to the published version of the manuscript.

Conflict of Interest Statement

The authors declare no conflict of interest

Funding Declaration

This research received no external funding.

Data Availability Statement

N/A

Ethical Declaration

N/A

Acknowledgments

We sincerely thank the experts for their professional evaluation and valuable recommendations, which have contributed to improving the quality of the experiment and the reliability of its results.

Declaration of generative AI in scholarly writing

Generative AI tools were used to assist in improving the clarity, grammar, and readability of the manuscript. The authors take full responsibility for the content of the work.

Abbreviations

The following abbreviations are used in this manuscript:

WSNs - Wireless Sensor Networks
EACP - Energy Aware Clustering Protocol
BS - Base Station
CH - Cluster Head
IoT - Internet of Things
TDMA - Time Division Multiple Access
HAC - Hierarchical Agglomerative Clustering
HAS - Hybrid Spectral-based Algorithms
LEACH - Low Energy Adaptive Clustering Hierarchy

Appendix A\B\C…, with appendix tile

None.

References

[1] Tian J. (2005). Software Quality Engineering. Testing, Quality Assurance, and Quantifiable Improvement. USA: John Wiley & Sons Inc. ISBN: 978-0-471-71345-6.
[2] Wiegers K., Beatty J. (2013). Software requirements. Third edition. USA: Microsoft Press. ISBN: 978-0-7356-7966-5.

[3] Combemale B., Gray J., Rumpe B. (2023). ChatGPT in software modeling. Software and Systems Modeling, 22, 777-779. DOI: 10.1007/s10270-023-01106-4.
[4] Marques N., Silva R.R., Bernardino J. (2024). Using ChatGPT in Software Requirements Engineering: A Comprehensive Review, Future Internet, 16(180), 1-21; DOI: 10.3390/fi16060180.
[5] Cheng H., Husen J.H., Lu Y., Racharak T., Yoshioka N., Ubayashi N., Washizaki H. (2025). Generative AI for Requirements Engineering: A Systematic Literature Review. Software: Practice and Experience, 56(2), 141-170. DOI: 10.1002/spe.70029.
[6] Pooley R., Wilcox P. (2003). Applying UML. USA: Advance. ISBN: 978-0750656832.
[7] Weiss B. (2011). Deductive verification of object-oriented software: dynamic frames, dynamic logic and predicate abstraction. Germany: KIT Scientific Publishing. ISBN: 978-3866446236.
[8] Drechsler R. (2017). Formal system verification : state-of the-art and future trends. Switzerland: Springer Nature Switzerland AG. ISBN: 978-3319576855.
[9] Kopp A., Orekhov S., Orlovskyi D. (2022). Towards Understandability Evaluation of Business Process Models using Activity Textual Analysis. CEUR Workshop Proceedings, Vol. 3312. 200-211. https://ceur-ws.org/Vol-3312/paper17.pdf.
[10] Baker P. (2025). ChatGPT For Dummies, For Dummies. USA: John Wiley & Sons. ISBN: 978-1394314454.
[11] Ekin S. (2023). ChatGPT. Prompt Engineering For ChatGPT: A Quick Guide To Techniques, Tips, And Best Practices. USA: OpenAI. DOI: 10.36227/techrxiv.22683919.
[12] Schaper N. (2024). Using ChatGPT to create constructively aligned assessment tasks and criteria in the context of higher education teaching. Artificial Intelligence for Quality Education. USA: IntechOpen. DOI: 10.5772/intechopen.1005129.
[13] Orekhov S., Taran P. (2024). Example of synthesizing a semantic kernel by CHATGPT. Proceedings of the XVII International Scientific and Practical Conference. Odessa, October 31 – November 1. Odessa, ONUT Publishing House. 576-578. https://otfk.od.ua/conference/files/17_01_2025_4.pdf.

Authors' Profiles

Sergey Orekhov is an Associate Professor in the Department of Software Engineering and Management Intelligent Technologies at National Technical University Kharkov Polytechnic Institute, Kharkiv, Ukraine. He is obtained his PhD from NTU KPI, Ukraine. His research area includes AI and SEO, AI in Marketing and WEB-oriented Systems.

Pavel Taran is a Postgraduate student in the Department of Software Engineering and Management Intelligent Technologies at National Technical University Kharkov Polytechnic Institute, Kharkiv, Ukraine. He is obtained his Master Degree from NURE, Ukraine. His research area includes AI in Marketing and WEB-oriented Systems.

Nikita Bahatsyi is a Postgraduate student in the Department of Software Engineering and Management Intelligent Technologies at National Technical University Kharkov Polytechnic Institute, Kharkiv, Ukraine. He is obtained his Master Degree from NTU KPI, Ukraine. His research area includes AI and WEB-oriented Systems.

How to cite this paper

Sergey Orekhov, Pavel Taran, Nikuta Bahatskyi, Yong Fan, "An Example of Developing a High-level Requirements Specification for AI-based Software using ChatGPT", International Journal of Wireless and Microwave Technologies(IJWMT), Vol.16, No.3, pp. 111-127, 2026. DOI:10.5815/ijwmt.2026.03.08